Q 1:Research and provide three real-world applications where C programming is extensively used, such as in embedded systems, operating systems, or game development.

A.1. Embedded Systems

Embedded systems are small computers inside everyday machines.
Examples include:

- Microcontrollers (ARM, PIC, AVR, Arduino)
- Home appliances (washing machines, microwave ovens, smart TVs)
- Automobiles (airbags, ABS, engine controllers)
- Medical devices (heart monitors, infusion pumps)
- IoT devices (smart sensors, smart home devices)

Why C is used:

- Extremely fast and efficient
- Uses very little memory
- Directly accesses hardware registers using pointers
- Portable across different chips and architectures

C is the dominant language in embedded programming because it controls hardware at a low level while staying readable and portable.

---

2. Operating Systems

Many modern operating systems are written mainly in C.
Examples include:

- UNIX – originally developed entirely in C
- Linux Kernel – mostly written in C
- Microsoft Windows – many core files and drivers use C
- macOS – based on Unix, uses C in low-level components
- Android OS components

Why C is used:

- Gives direct memory and hardware control
- Fast execution required for kernel operations
- Allows system-level programming (drivers, file systems, process control)
- Portable across hardware platforms

C remains the best choice for building OS kernels and drivers due to its performance and low-level access.

---

3. Game Development & Game Engines

Although high-level languages are used for game logic, the core performance parts of games are still written in C/C++.

Examples include:

- Game engines:
  - Unreal Engine
  - Unity (engine core)
  - Godot Engine (C++ core)
- Graphics libraries:
  - OpenGL
  - DirectX
  - Vulkan
- High-performance gaming applications

Why C is used:

- High speed and optimized performance
- Allows direct access to GPU and memory
- Handles real-time physics, rendering, audio, input
- Efficient for creating game engines and graphics systems

C provides the speed required for real-time graphics rendering and smooth gameplay.

Q 2:Install a C compiler on your system and configure the IDE. Write your first program to print "Hello, World!" and run it.

A.#include <stdio.h>
int main() {
    printf("Hello, World!");
    return 0;
}

Q 3:Write a C program that includes variables, constants, and comments. Declare and use different data types (int, char, float) and display their values.

A. #include <stdio.h>   // Header file

// This is a constant declaration
#define PI 3.14
int main() {

    // Variable declarations

    int age = 20;   // integer variable
    char grade = 'A';   // character variable
    float salary = 45000.75; // float variable

    // Displaying the values

    printf("Value of integer variable age = %d\n", age);
    printf("Value of character variable grade = %c\n", grade);
    printf("Value of float variable salary = %.2f\n", salary);

    // Displaying constant

    printf("Constant PI = %.2f\n", PI);
    return 0; // End of program
}

Output (when program is executed)

```
Value of integer variable age = 20
Value of character variable grade = A
Value of float variable salary = 45000.75
Constant PI = 3.14
```

Q 4:Write a C program that accepts two integers from the user and performs arithmetic, relational, and logical operations on them. Display the results.

A.#include <stdio.h>

```
int main() {

    int a, b;

    // Taking input from the user
    printf("Enter first integer: ");
    scanf("%d", &a);
    printf("Enter second integer: ");
    scanf("%d", &b);

    // Arithmetic operations
    printf("\n--- Arithmetic Operations ---\n");
    printf("Addition (a + b) = %d\n", a + b);
    printf("Subtraction (a - b) = %d\n", a - b);
    printf("Multiplication (a * b) = %d\n", a * b);
    printf("Division (a / b) = %d\n", a / b);
    printf("Modulus (a %% b) = %d\n", a % b);



    // Relational operations
```

```
    printf("\n--- Relational Operations ---\n");
    printf("a > b : %d\n", a > b);
    printf("a < b : %d\n", a < b);
    printf("a == b: %d\n", a == b);
    printf("a != b: %d\n", a != b);
    printf("a >= b: %d\n", a >= b);
    printf("a <= b: %d\n", a <= b);

    // Logical operations
    printf("\n--- Logical Operations ---\n");
    printf("(a > 0) && (b > 0) = %d\n", (a > 0) && (b > 0));
    printf("(a > 0) || (b > 0) = %d\n", (a > 0) || (b > 0));
    printf("!(a > 0) = %d\n", !(a > 0));

    return 0;
}
```

Sample Output

```
Enter first integer: 10
Enter second integer: 5

--- Arithmetic Operations ---
Addition (a + b) = 15
Subtraction (a - b) = 5
Multiplication (a * b) = 50
Division (a / b) = 2
Modulus (a % b) = 0

--- Relational Operations ---
a > b : 1
a < b : 0
```

```
a == b: 0
a != b: 1
a >= b: 1
a <= b: 0

--- Logical Operations ---
(a > 0) && (b > 0) = 1
(a > 0) || (b > 0) = 1
!(a > 0) = 0
```

Q 5 : Write a C program to check if a number is even or odd using an if-else statement. Extend the program using a switch statement to display the month name based on the user's input (1 for January, 2 for February, etc.).

A.

```
#include <stdio.h>
int main() {
    int num, month;

    // ---------- Even or Odd Check ----------
    printf("Enter a number: ");
    scanf("%d", &num);
    if (num % 2 == 0) {
        printf("%d is Even.\n", num);
    } else {
        printf("%d is Odd.\n", num);
    }

    // ---------- Month Name Using Switch ----------
    printf("\nEnter month number (1-12): ");
    scanf("%d", &month);

    switch(month) {
```

```
        case 1:  printf("January\n"); break;
        case 2:  printf("February\n"); break;
        case 3:  printf("March\n"); break;
        case 4:  printf("April\n"); break;
        case 5:  printf("May\n"); break;
        case 6:  printf("June\n"); break;
        case 7:  printf("July\n"); break;
        case 8:  printf("August\n"); break;
        case 9:  printf("September\n"); break;
        case 10: printf("October\n"); break;
        case 11: printf("November\n"); break;
        case 12: printf("December\n"); break;

        default:
            printf("Invalid month number!\n");
    }

    return 0;
}
```

Sample Output

```
Enter a number: 15
15 is Odd.

Enter month number (1-12): 4
April
```

Q 6:Write a C program to print numbers from 1 to 10 using all three types of loops (while, for, do-while).

A. a.<u>While Loop</u>

```
#include<stdio.h>
 main(){
```

```
        int i=1;
        while(i<=10){
        printf("\n i==%d" ,i);
        i++;
      }
  }
```

b.For Loop

```
      #include<stdio.h>
       main(){
       int i;
       for(i=1;i<=10;i++){
       printf("\n i==%d" ,i);

     }
  }
```

c.do-while Loop

```
       #include<stdio.h>
       main(){
       int i=1;
      do{
      printf("\n i==%d" ,i);
      i++;
   } while(i<=10);
}
```

Q 7:Write a C program that uses the break statement to stop printing numbers when it reaches 5. Modify the program to skip printing the number 3 using the continue statement.

A.

```
#include <stdio.h>
int main() {
    int i;
    for (i = 1; i <= 10; i++) {
        if (i == 5) {
            break;   // Stop the loop when i becomes 5
        }
        printf("%d ", i);
    }
    return 0;
}
```

```
#include <stdio.h>
int main() {
    int i;
    for (i = 1; i <= 10; i++) {
        if (i == 3) {
            continue;   // Skip printing 3
        }
        printf("%d ", i);
    }
    return 0;
}
```

Q 8:Write a C program that calculates the factorial of a number using a function. Include function declaration, definition, and call.

A.

```
    #include<stdio.h>
int factFind(int num){
    int f;
    if(num==1){
        return 1;
    }
    f=num*factFind(num-1);//f=5*factFind(4)
    return f;
}
main(){
    printf("\n factorial=%d",factFind(7));
}
```

Q 9:Write a C program that stores 5 integers in a one-dimensional array and prints them. Extend this to handle a two-dimensional array (3x3 matrix) and calculate the sum of all elements.

A.1. One-Dimensional Array (Store and Print 5 Integers)

Program

```
#include <stdio.h>
int main() {
```

```
    int arr[5] = {10, 20, 30, 40, 50};
    int i;
    printf("Elements of 1D array:\n");
    for (i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

2. Two-Dimensional Array (3×3 Matrix) and Sum of Elements

Program

```
#include <stdio.h>
int main() {
    int matrix[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };
    int i, j, sum = 0;
    printf("Elements of 3x3 matrix:\n");
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            printf("%d ", matrix[i][j]);
            sum += matrix[i][j];
        }
        printf("\n");
    }
    printf("Sum of all elements = %d", sum);
    return 0;
}
```

Q 10: Write a C program to demonstrate pointer usage. Use a pointer to modify the value of a variable and print the result

A.

```
#include<stdio.h>
 main(){
  int a=10;
  int *ptr;
   ptr=&a;
   printf("\n address=%p & value=%d",ptr,*ptr);
```

```
    *ptr=100;
    printf("\n a=%d",a);
 }
```

Q 11:Write a C program that takes two strings from the user and concatenates them using strcat(). Display the concatenated string and its length using strlen().

A.

```
#include <stdio.h>
#include <string.h>

int main() {
char str[50], str1[50], str2[50];
printf("\n Enter The string:");
scanf("%s",str1);
printf("\n Enter The string:");
scanf("%s",str2);
strcat(str1,str2);
printf("\n After concating str1 and str2 str is %s",str);
strlen(str);
printf("\n Length of str is %d",strlen(str));
return 0;
}
```

Q 12:Write a C program that defines a structure to store a student's details (name, roll number, and marks). Use an array of structures to store details of 3 students and print them.

A.

```
#include<stdio.h>
struct Student{
char name[20];
 int roll number;
 float marks;
};
int main(){
   struct Student s[3];
   int i;
   for(i=0;i<3;i++){
      printf("\n Enter name, roll number & marks of student %d:",i+1);
      scanf("%s %d %f",s[i].name,&s[i].roll number,&s[i].marks);
   }
   for(i=0;i<5;i++){
      printf("\n name=%s",s[i].name);
```

```
        printf("\t roll number=%d",s[i].roll number);
        printf("\t marks=%.2f",s[i].marks);
    }
}
```

Q 13:Write a C program to create a file, write a string into it, close the file, then open the file again to read and display its contents.

A.

```
#include<stdio.h>
    int main(){
    char data[20];
    FILE *fp;
    fp = fopen("hello.rtf","w");
    fprintf(fp,"hello world");
    fclose(fp);
    fp = fopen("hello.rtf","r");
    fgets(data,20,fp);
    printf("\n reading data from file=%s",data);
    fclose(fp);
}
```